

Week 7 Applied Class Worksheet Solutions

COMP2048

Dr. Kasra Khosoussi

Semester 1, 2026

Task 1.

For each of the following languages, determine whether it is Turing-decidable and/or Turing-recognizable. Justify your answers.

1. $\text{HALT}_{2048} = \{\langle M, w \rangle \mid \text{TM } M \text{ halts on input } w \text{ within 2048 steps}\}$
2. $A_{2048} = \{\langle M, w \rangle \mid \text{TM } M \text{ accepts input } w \text{ within 2048 steps}\}$

Solution.

We analyze each language by asking whether there is a Turing machine that always halts and decides membership.

1. **HALT_{2048} is Turing-decidable, and therefore also Turing-recognizable.**

On input $\langle M, w \rangle$, a decider can simulate M on input w for at most 2048 steps.

- If M halts within those 2048 steps, accept.
- If M has not halted after 2048 steps, reject.

This procedure always halts after a finite, fixed number of simulation steps, so it is a decider.

Since every decidable language is also recognizable, HALT_{2048} is also Turing-recognizable.

2. **A_{2048} is Turing-decidable, and therefore also Turing-recognizable.**

On input $\langle M, w \rangle$, simulate M on w for at most 2048 steps.

- If M enters an accept state within those 2048 steps, accept.
- Otherwise, reject.

Again, this algorithm always halts because the simulation is bounded by 2048 steps.

Therefore A_{2048} is decidable, and hence also recognizable.

Task 2.

Someone claims to have constructed a decider H for the acceptance problem

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM that accepts } w\}.$$

Their construction is as follows:

$H = \text{“On input } \langle M, w \rangle\text{:”}$

1. *Simulate M on input w .*
2. *If M accepts, **accept**.*
3. *If M rejects, **reject**.*
4. *Otherwise, if M does not halt on w , conclude that it will never accept and **reject**.”*

Identify and clearly explain the flaw in their argument. Why is H not a valid decider?

Solution.

The flaw is in step 4. While simulating M on input w , if M never halts, then the simulation also runs forever. The machine H has no effective way to detect, after finite time, that M will *never* halt.

In more detail:

- If M accepts, then H can eventually see this and accept.
- If M rejects, then H can eventually see this and reject.
- But if M loops forever, H will also keep simulating forever.

The statement “if M does not halt on w , conclude that it will never accept” is exactly the hard part. H is *not* a decider, because a decider must halt on every input. For inputs where M loops forever on w , the machine H also loops forever and never reaches an accept/reject decision.

Task 3.

Suppose we know that the Halting Problem is Turing-undecidable. Use reduction to prove that the Acceptance Problem A_{TM} is also Turing-undecidable.

Hint: Assume that A_{TM} has a decider A . Show how to construct a decider for the Halting Problem using A , and derive a contradiction.

Solution.

We prove this by reduction from the Halting Problem.

Assume, for contradiction, that A_{TM} is decidable. Then there exists a decider

$$A(\langle M, w \rangle)$$

that accepts exactly when M accepts w , and rejects otherwise. We will use A to decide the Halting Problem

$$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \},$$

which is impossible since we know $HALT_{TM}$ is undecidable.

Given an input $\langle M, w \rangle$ to $HALT_{TM}$, construct a new Turing machine N_w such that:

$N_w(x)$ ignores its input x , simulates M on w , and behaves as follows:

- If M ever halts on w (either accepting or rejecting), then N_w accepts.
- If M never halts on w , then N_w also runs forever.

Now consider the pair $\langle N_w, \varepsilon \rangle$, where ε is the empty string.

- If M halts on w , then N_w accepts ε , so $\langle N_w, \varepsilon \rangle \in A_{TM}$.
- If M does not halt on w , then N_w never accepts ε , so $\langle N_w, \varepsilon \rangle \notin A_{TM}$.

Therefore, if we run the decider A on $\langle N_w, \varepsilon \rangle$, we can determine whether M halts on w :

- If A accepts, answer that M halts on w .
- If A rejects, answer that M does not halt on w .

This gives a decider for $HALT_{TM}$, contradicting the fact that the Halting Problem is undecidable. Hence our assumption was false and A does not exist.

Task 4.

Prove that the class of Turing-recognizable languages is closed under union.

Solution.

Let L_1 and L_2 be Turing-recognizable languages. We must show that

$$L_1 \cup L_2$$

is also Turing-recognizable.

Since L_1 and L_2 are recognizable, there exist Turing machines M_1 and M_2 such that:

- M_1 recognizes L_1 ,
- M_2 recognizes L_2 .

This means:

- if $w \in L_1$, then M_1 accepts w after some finite time;
- if $w \notin L_1$, then M_1 may reject or loop forever;
- similarly for M_2 and L_2 .

We now construct a recognizer M for $L_1 \cup L_2$.

On input w , machine M does the following:

1. Simulate M_1 on w for one step.
2. Simulate M_2 on w for one step.
3. Simulate M_1 on w for one more step.
4. Simulate M_2 on w for one more step.
5. Continue in this alternating way.
6. If either M_1 or M_2 ever accepts, then accept.
7. If both machines reject, then reject.
8. Otherwise continue the simulation (until both machines halt, or forever).

Why does this work?

- If $w \in L_1 \cup L_2$, then $w \in L_1$ or $w \in L_2$.
- If $w \in L_1$, then M_1 eventually accepts w , so during the “parallel” step-by-step simulation, M will eventually see that acceptance and accept.
- If $w \in L_2$, the same reasoning applies using M_2 .

So every string in $L_1 \cup L_2$ is accepted by M . If $w \notin L_1 \cup L_2$, then neither M_1 nor M_2 ever accepts w . In that case, M may reject (if both machines reject) or run forever which is allowed for a recognizer. Therefore M recognizes $L_1 \cup L_2$, so the union of two Turing-recognizable languages is again Turing-recognizable.